

Kubernetes Certified Application Developer

Your Name

Date:

Contents

Chapter 1

Notes info

Purpose of notes There purpose of these notes are to gain an understanding of Docker and have reference material in case I decide to sit the DCA exam.

Chapter 2

Introduction

2.1 Kubernetes Architecture

Nodes A node is a machine, physical or virtual upon which Kubernetes is installed. This worker machine is where containers will be launched by Kubernetes.was known as minions in the past... These terms may be used interchangeably.

Consider a node failing.....application will go down.....hence we need multiple nodes.

Cluster A cluster is a group of nodes grouped together. This way when one node fails, your application is still accessible from the other nodes. Having multiple nodes helps in sharing the load as well.

Master node Things to consider - What is responsible for managing the cluster - Where is the information about members of the cluster stored? - How are nodes monitored? - When a node fails, how do you move the workload of the failed node to another worker node?

This is the purpose of the master. The master is another node with Kubernetes installed in it and is configured as a master. The master watches over the nodes in the cluster and is responsible for the actual orchestration of containers on the worker nodes.

2.1.1 What is installed on a system when installing Kubernetes?

- API Server
- etcd
- Kubelet
- Container Runtime
- Controller
- Scheduler

API Server

An API server which acts as the frontend for Kubernetes. The user management devices, command line interfaces all talk to the API server to interact with the Kubernetes cluster.

etcd keystore

Etcd is a distributed, reliable key value store used by Kubernetes to store all data used to manage the cluster. The way to think of it is, when you have a multiple nodes and masters in your cluster, etcd stores all that information on all the nodes in the cluster in a distributed manner Etcd is responsible for implementing locks within the cluster to ensure that there are no conflicts between the masters,

The Scheduler

is responsible for distributing work or containers across multiple nodes. It looks for newly created containers and assigns them to nodes.

Controllers

The "brain" behind orchestration. They are responsible for noticing and responding when nodes, containers or endpoints go down. The controllers decide whether to bring up new containers.

Container Runtime

They are responsible for noticing and responding when nodes, containers or endpoints runtime is the underlying software that is used to run containers. An example is Docker, but alternatives exist.

Kubelet

This is the agent that runs on each node in the cluster. The agent is responsible for making sure that the containers are running on the nodes as expected.

There are two types of server, masters and a worker along with a setup of components which make up Kubernetes.

How does one server become a master and the other the slave.

The worker node, or minion as it is also known, is where the containers are hosted.

The master has the Kube API server and that is what makes it a master. Similarly the worker nodes have the Kubelet agent that is responsible for interacting with master to provide both health information of the worker node and carry out actions requested by the master on the worker nodes.

All the information gathered is stored in a key value store on the master. The key value store is based on the popular etcd framework.

The master also has the control manager and the scheduler.

Kube command line or kubectl or kube control

The kube control tool, is used to deploy and manage applications on Kubernetes cluster to get cluster information, to get the status of other nodes in the cluster. And to manage other things. The kubectl cluster info command is used to view information about the cluster and the kubectl get nodes command is used to list all the nodes as a part of the cluster.

2.2 Docker-vs-ContainerD

Initially kubernetes was the container runtime for docker only. Support for other container runtimes was added. This was done via an interface called Container Runtime Interface or CRI.

CRI allows any vendor to work as a container runtime for Kubernetes, as long as they adhere to the OCI standards. OCI stands for the open container initiative and it consists of an image spec and runtime spec.

Image spec means the specifications on how an image should be built. An image spec defined the specifications on how an image should be built and the runtime spec defined the standards on how nay container runtime should be developed.

Keeping the OCI standards in mind. Anyone can build a container runtime that can be used by anybody to work with Kubernetes Note: Docker wasn't built for CRI. To maintain support between kubernetes and Docker, Kubernetes introduced dockershim. Docker runs without CRI.

Docker isn't just a container runtime along. Docker consists of multiple tools that are put together. - Docker CLI, the Docker API and the build tools that help in building images. There was support for volumes or security. There is a container runtime called run C and that managed a container called container D Container D can be used as a runtime on its own separated from Docker.

...dockershim and docker support was removed from kubernetes but docker follows the OCI standards now.

Container D is a part of Docker but is a separate project on its own. You can run container D without having to install Docker itself. If you don't need dockers other features you could ideally just install container D alone.

Chapter 3

Core Concepts

Chapter 4

Configuration

Chapter 5

Multi-Container Pods

Chapter 6

Observability

Chapter 7

POD Design

Chapter 8

Services and Networking

Chapter 9

State Persistence

Chapter 10

Security

Chapter 11

Helm Fundamentals

Chapter 12

Kustomize